

PX457 Assignment 5

Introduction

Your task in this assignment is to implement and accelerate a simulation code using multiple forms of parallelism and undertake some performance measurements to understand the efficacy of your solution under different conditions. You will measure the performance of the algorithm, optimize its serial portions, and choose an appropriate strategy to implement parallelism on up to 96 cores of Avon. **The assessment in this exercise is split roughly threefold between code quality, serial and parallel performance, and writing a full report on the results.** Please note that it is possible to write a good report even if the performance of your code is not as good as you might have hoped, so do not skimp on that section or leave it too late, as much of the report-writing is easiest if done as you go along. Your job in the report is to explain what you have done to modify the code and why, and to demonstrate that you understand how you have improved its performance.

Code is provided implementing an unoptimised, serial starting point, which you are expected to modify in any way you see fit, providing that it retains the capabilities and flexibility of the original code. Please use your judgement to interpret what "retaining the capabilities of the original code means": We do not want to field questions along the lines of 'are we allowed to do [X]?'.

You **must** use Avon for final timing measurements for this assignment, though initial development and testing can be done on vonneuman, stelvio or the SCRTP Taskfarm. Scripts are provided which submit to the PX457 partition on Avon. **You must not run your code on hammer or the Avon login node. Failure to observe this could result in suspension of your SCRTP accounts..**

System: Gravitational Dynamics

The subject of this assignment is a code implementing a gravitational (N+1)-body simulation modelling stars in a globular cluster in the vicinity of a supermassive black hole. Note, the astrophysics in this code is likely not at all realistic: We are *not* astrophysicists, and have simply made up reasonable parameters of the system to suit the exercise. The system obeys Newtonian gravity, and the code is designed to simulate the evolution of the system over time. The black hole at the origin of coordinates has an appropriate mass and event horizon radius.

You are provided with some code to get you started. This defines (in `data.c`) some initial conditions for the simulation and the basic simulation structure (in `gravity.c`). The model operates in a Cartesian coordinate system with its origin at the centre of the black hole. We assume the black hole is a sphere with its radius being the event horizon. The black hole does not move in this simulation, and is imagined to be very much more massive than the stars so that their effect on it can be neglected: this is a significant simplification of the problem and not very realistic, but it makes the simulations much easier to keep stable. Several important arrays are populated to hold data about the state and kinematics of the stars in the globular cluster:

- `active[i]` - 1 if star is outside event horizon or within simulation region, 0 otherwise
- `mass[i]` - m_i mass of the i^{th} star.
- `radius[i]` - s_i radius of the i^{th} star.
- `pos[3*i+k]` - r_i^k position of the i^{th} star ($k = 0, 1, 2$ for x, y, z).
- `vel[3*i+k]` - v_i^k velocity of the i^{th} star ($k = 0, 1, 2$ for x, y, z).
- `acc[3*i+k]` - a_i^k acceleration of the i^{th} star ($k = 0, 1, 2$ for x, y, z).

The simulation consists of `nStars` stars which interact with each other (and are acted on by the black hole) by gravitational attraction. We neglect collisions, though if you were to use the array `radius` you could include them, but they would be exceedingly rare.

The position and velocity of each star is evolved over a number of time steps using the Velocity-Verlet (VV) algorithm. This requires only a single copy of each position, velocity and acceleration to be held in memory at any time. Brackets denote the times at which each quantity is stored and the code loops over `Nsteps` iterations of the update scheme. It is necessary to use an appropriately small timestep Δt to ensure the stability of the simulation: suitable values in these units are between 10^{-8} and 10^{-4} time units, depending on the orbital parameters. The VV algorithm is a second-order integrator, and is used to update the positions and velocities of the stars at each timestep. The algorithm is as follows:

$$v_i^k(t + \frac{1}{2}\Delta t) = v_i^k(t) + \frac{1}{2}a_i^k(t)\Delta t \quad (1)$$

$$r_i^k(t + \Delta t) = r_i^k(t) + v_i^k(t + \frac{1}{2}\Delta t)\Delta t \quad (2)$$

$$v_i^k(t + \Delta t) = v_i^k(t + \frac{1}{2}\Delta t) + \frac{1}{2}a_i^k(t + \Delta t)\Delta t \quad (3)$$

The updated acceleration of the i^{th} star is recomputed from the current positions after step (2) using Newtonian gravitation. Here it is convenient to arrange the three components r_i^k into a position vector $\mathbf{r}_i$, and to express the vector separating star i and j as $\mathbf{r}_{ij}$.

$$a_i^k = -r_i^k \frac{GM_{BH}}{|\mathbf{r}_i|^3} + \sum_{j \neq i} (r_j^k - r_i^k) \frac{Gm_j}{|\mathbf{r}_{ij}|^3} \quad (4)$$

The sum runs over all stars other than the i^{th} . We also add a very crude approximation to drag resistance for stars which have entered the vicinity of the black hole and are subject to the clouds of dust nearby (crudely assuming the dust is stationary with respect to the black hole!). This has been coded for you already but its magnitude is set to zero as it is quite easy to destabilise the simulation with it - you can try it reactivated if you like, but even if left set to zero, its calculation should not be disabled. The drag term is given by:

$$F_{i,drag}^k = -\frac{1}{2}\rho_{dust}v_i^k|\mathbf{v}_i|A_i \quad (5)$$

where ρ_{dust} is the density of the dust, A_i is the cross-sectional area of the star, and $\mathbf{v}_i$ is the velocity of the star. The drag term is added to the acceleration of the star in the direction opposite to its velocity.

Various constants are defined within the skeleton code and do not need to be changed, including the mass of the black hole M_E (BHMmass) set to $200000M_{\text{sun}}$, the event horizon radius (BHRradius) and the gravitational constant G (bigG).

The code also provides a suitable loop over time. At each step it checks if any stars have reached the black hole's event horizon, or have escaped the simulation by reaching a distance of 200000 light years from the black hole, reporting these to screen and to `output.dat`. In testing, we have not yet managed to legitimately get a star to collide with the black hole without setting it up to do so, but you might be able to do so with a large enough number of stars and a long enough simulation time!

1 Getting Started

First, visit the Assignment 5 section of the Moodle page and download `gravity.zip`, uncompressing into your space on the SC RTP system in the usual way. The code as provided will compile and run, as before we have provided you with a `Makefile` to ease compilation of the code, but it does not implement a useful simulation. Running the resulting `gravity` executable should be very fast, and can hence be tested interactively on `vonneumann` or `stelvio`). The program provides several command line arguments to configure the simulation. You can find out what these are by running `./gravity -h`:

```
$ ./gravity -h
Usage: ./gravity [options]
Options:
  -N <nStars>          Number of stars
  -S <nSteps>           Number of steps
  -D <timestep>         Timestep (in Myr)
  -K <KE_init>          Initial kinetic energy
  -M <ClusterMass>      Cluster mass (in MSun)
  -A <Apogee>           Cluster apogee (in Ly)
  -P <Perigee>          Cluster perigee (in Ly)
  -R <Radius>           Cluster radius (in Ly)
  -t                   Use time-based random seed
  -h                   Show this help message
```

Full details on these arguments and what they do are covered in the Appendix. **Do not** try running with more than one or two stars just yet or trying other parameters! Even though it doesn't do a lot, the main loops are in place, and it can run for a reasonable time.

2 Serial Implementation

Start by examining the code and **make sure you understand the data structures and organization** in `data.c` and `gravity.c`. In particular, the function `initialiseCluster()`

populates the kinematic arrays described earlier with initial conditions for the simulation, so check you understand how the initial orbital parameters are calculated so you can check your later results make sense. Without supplying any arguments, the code will generate just 1 star and print some useful information on startup and on a subset of timesteps.

Your first task is to implement the (serial) VV algorithm and evolve the star over time using **just** the acceleration of the black hole's gravity:

- Implement appropriate loops to generate the action of equations (1) to (3), on the `pos`, `vel` and `acc` arrays. The comments in the code should help with this.
- Implement the first term in equation (4) for the acceleration due to the black hole.
- Add print statements which output the x , y and z coordinates of the star every s steps, so you can plot the orbit and convince yourself it is sensible. You should adapt s and the total simulation steps run (via the `-S` command line argument to produce a reasonable plot of the path).
- Include a plot showing this orbit in the x, y plane as part of your report together with the simulation parameters used.

As implemented to this point, we have not included the interactions between the stars. Try running with 5 stars and convince yourself this is the case. Your second task is to include these interactions. You should still be able to run on `vonneumann` or `stelvio` at this point:

- Implement the second term in equation (4) for the star-star interactions. Again, the comments should show the appropriate location, and a double loop over the stars will be needed.
- Run with 5 stars and plot their trajectories before and after you added the interactions and convince yourself these are sensible. Including these plots in your report, along with the parameters used.

You may notice the “Bookkeeping” code following the VV algorithm in `gravity.c` checks whether a star has impacted the black hole or escaped from the simulation region, setting its entry in the array `active` to zero if it has.

- Modify your code for implementing the velocity Verlet algorithm such that stars flagged as inactive in this fashion no longer have their position updated.
- However, make sure their mass and position are still being used when computing the gravitational acceleration of all other stars.
- In practice it can be quite tricky to set up the simulation so that stars will impact/escape, but experiment with the cluster apogee/perigee arguments to see if you can get this to happen (but don't worry if you cannot!).

Once satisfied that your code correctly implements all of this, save a copy of your `gravity.c` file at this point as `gravity_serial.c` and keep it safe for submission later.

3 Parallel Implementation

We should now have the basic physics and numerical methods coded up, but a typical globular cluster will contain many more than 5 stars (thousands to millions). Our serial implementation simply won't scale to a problem of that size as we have $O(N^2)$ complexity. Your next task is thus to parallelize and optimize the program so that it can be run at scale on the Avon system. As before, you should copy your code to Avon so you can compile and run on the batch system there, though you can develop and test with 5 or so stars on `vonneumann`, `stelvio` or by submitting jobs to the SCRTP Taskfarm (Refer back to the material in Workshops if you need a refresher). **Remember that your home area on Avon is NOT backed up, so you should get into the habit of copying work to and from the SCRTP desktop system. See Workshop 3 for details.** As already stated, be very careful about where and how you run your code! You should run by submitting jobs to the PX457 partition on Avon except for the small scale tests noted above. A script for use in the PX457 partition is provided in the `gravity.zip` file. It will need modifying for your choice of parallel strategy.

Your first task is to accurately time various segments of the code to determine where the time is spent. Remember that the distribution of time within different parts of the code will change as the number of stars increases.

- Add timers around the various loops in the time-stepping to understand their relative importance.
- Document their relative importance and how this changes as the number of stars in the simulation changes (use your judgement on the range of numbers to use, bearing in mind overall runtime).
- You may need to comment out print statements you added in the first part of the exercise to get accurate timings. However, do not remove statements that were there originally as we expect the original output to be retained.

Using this information and ongoing measurements, it is now your task to parallelize the code and optimize any remaining serial parts using what you have learned to this point in the course and previous assignments:

- You may use OpenMP, MPI, or a combination thereof to parallelise your code, but be aware that top marks are likely to be most readily available to hybrid OpenMP/MPI solutions. Your report should **outline the steps needed and justify the choice of strategy**.
- You should identify opportunities for serial optimization in the remaining serial portions and implement and **justify these**, bearing in mind that **the mark scheme takes into account code quality, readability, and portability**.

It may be the case that the most efficient serial algorithms or data structuring may not scale well to higher processor counts, so you will need to balance your approaches.

- You should add statements to **accurately** time regions of your code using `omp_get_wtime()`, `MPI_Wtime()` or equivalent. If necessary, consult Assignment 3

and Assignment 4 code for example timings code. Consider the effect of synchronisation on your timings, and discuss in your report what measures you have taken to ensure accurate timing.

- You will need to update the `Makefile` and Slurm scripts as appropriate for your choice of parallelization/optimization.
- Your report must **include evidence that your code is still functioning correctly** after parallelization and optimization. For example, you could plot some the distance of the nearest object from the black hole under a range of different initial conditions producing different behaviour, and discuss the stability as you change simulation or parallel parameters.

Your code must be capable of running on any number of processors, up to $P = 48$ for a pure OpenMP code, or $P = 96$ for an MPI or MPI/OpenMP code. You should try to parallelise and optimize as much of the code as possible, however we want you to avoid changing code in `data.c` unless you feel the need to restructure the simulation data (in that case, you **must** explain and justify the changes). You **must not** modify the random number generator code. Some further requirements are detailed below:

- The parallel code **must** generate and use the same initial conditions as the serial code provided.
- The format of all output to screen and to `output.dat` **must** match the serial code. This will likely mean ensuring that only one thread/task calls `printf` and `fprintf`, and that any output you write yourself goes to a file.
- The parallel code should ideally give the **same numerical output** on any number of processors/threads, including a single processor/thread. This needs thought to achieve in practice, as it requires consistent ordering of floating point operations. Small differences in rounding due to different ordering can lead to dramatically different final results.
- You will lose marks for writing code that could result in a race condition or a deadlock situation, even if in practice they do not seem to occur very often.
- You may **not** assume that the gravitational interaction is zero beyond some distance. You must compute interactions between all bodies at every step as per equation (4).

A sensible number of stars with which to develop your parallel code is 500, using 4 processors/threads. However keep in mind that the parallel implementation which performs best at this system size may not be optimal for larger systems and processor counts. Your report must **detail the parallelism and serial optimization strategies you've chosen, giving reasons why you think this is appropriate/efficient**. Describe any problems you've had implementing the simulation in parallel.

4 Measuring and Reporting Performance Metrics

Your report's justification of the chosen parallelism/optimization strategy you've chosen must be backed up with relevant performance measurements

Using timings gathered from your final program running on *Avon*, measure both the speedup ψ and the Karp-Flatt metric F (experimentally determined serial fraction) as

a function of P . Here, P is the number of OpenMP threads, MPI tasks, or product thereof if you have written a hybrid code. For both ψ and F , plot them against P for `nStars= 500, 600 and 750` on the same axes. **Include these two plots in your report.**

You *do not* need to take average timings over several runs - the loop over time steps effectively averages for you. You may assume the computational work N is a constant during the simulation (a very good approximation) and should take measurements for as many P values as you can. If you have implemented a hybrid solution, you should consider how these change as you keep P constant whilst varying the number of OpenMP threads and MPI processes. Gathering this data may take some time, so **please don't leave it until just before the deadline.**

Discuss these plots in your report. Specifically, determine from your Karp-Flatt data if the strong scaling of your code is likely to be limited by sections which have not been parallelised, or by parallel overheads such as inter-processor communication and thread management. Your conclusions may depend on the system size.

If it has not turned out to be efficient under all circumstances, that is fine, but **you should understand and discuss the reasons why not in your report**, with reference to the above performance metrics.

Running Full Scale Simulations

A suitable size for full scale simulation is 1,500 stars, with all other parameters left at their defaults. Your code should be capable of simulating this many stars for 100000 steps, **within the five minute walltime of the PX457 partition.** Your report should state how long this simulation takes with your code. As a rough guideline, a pure OpenMP solution using 24 cores on a single node can complete this simulation in under a minute or two. A solution which can use 96 cores in the PX457 partition (employing MPI) can complete in less than one minute. **Do not panic if your code isn't quite this fast!** Substantial credit will be awarded for any **correct** parallel code that compiles without warning, the accompanying performance measurements, and sensible discussion.

You need only run this full scale simulation once for timing purposes, but as long as the queue is not too busy feel free to try and optimise the code further.

Deadline and Submission

You have until noon on 9th January 2026 to complete the assignment. Please get started before the end of term - we will have very limited ability to respond to questions and queries during the holiday, and uninterrupted SCRTP service cannot be guaranteed due to staff absence. Our strong advice is to attempt to make substantial progress on the assignment during term time.

To submit the assignment, zip all the files needed to compile and run the program, along with your report, into a single a zip file with the same name as your user code, for example like this

```
zip phuxyz.zip assignment5/* report.pdf
```

where you use your user code instead of `phuxyz` and I have assumed all your files are in the directory `assignment5/`. Make sure you include your `gravity_serial.c` file from part 1.

Don't submit object files, the executable, output results or scaling plots as supplementary files - any plots should be included in the report. Upload the zip file to the Moodle submission page as usual.

Appendix: Command Line Arguments

These are the command line arguments accepted by the program. Be careful with these: with everything set to the defaults and varying $-N$, the program will run stably for quite a long time without any stars escaping. With variation, particularly to the perigee of the cluster, you will need to correspondingly update the timestep to keep the simulation stable.

Argument	Purpose	Syntax & Example
$-t$	If present, uses current time as random seed (Default: false)	$-t$./gravity $-t$
$-N$	Sets the number of stars (Default: 1)	$-N$ <val> ./gravity $-N$ 1500
$-S$	Sets the number of steps to run for (Default: 100000)	$-S$ <val> ./gravity $-S$ 10
$-D$	Sets the timestep (Default: 0.00001)	$-D$ <val> ./gravity $-D$ 0.0001
$-K$	Sets the initial KE of the cluster in $M_{\text{sun}} * \text{Ly}^2 / \text{My}^2$ (Default: 0)	$-M$ <val> ./gravity $-N$ 2 $-M$ 500000
$-M$	Sets the Mass of the cluster in M_{sun} (Default: 0)	$-M$ <val> ./gravity $-N$ 4 $-M$ 4
$-A$	Sets the apogee of the cluster in Ly (Default: 10)	$-A$ <val> ./gravity $-A$ 200
$-P$	Sets the perigee of the cluster in Ly (Default: 10)	$-P$ <val> ./gravity $-P$ 2
$-R$	Sets the radius of the cluster in Ly (Default: 1)	$-R$ <val> ./gravity $-R$ 5

Examples:

- To generate 1 star, iterating for 10 timesteps:
./gravity $-N$ 1 $-S$ 10
- To generate a system of 2 widely spaced light stars (whose mass can be neglected compared to the black hole), iterating for 10 timesteps:
./gravity $-N$ 2 $-R$ 10 $-S$ 10 $-M$ 2 (useful for testing the stability of the integration).
- To run with the specified final settings for the scaling tests:
./gravity $-N$ 1500
(leaves number of steps and initial KE at default values).
- To run with 100 stars, total mass 100 M_{sun} , cluster apogee 10 Ly, perigee 2 Ly, radius 0.2 Ly for 1000000 steps:

`./gravity -N 100 -M 100 -A 10 -P 2 -R 0.2 -S 1000000` (try plotting the behaviour of the distance from the nearest star to the black hole over time for this one, varying timestep and keeping total simulation time the same).